
Lasso Regression Solution

MSCS2202 – Challenge Assignment

JIAJIE LI

1. TL;DR

This report documents the end-to-end machine learning pipeline used to predict continuous target values from 15 numerical features (x0–x14) using Lasso Regression. The evaluation metric is R^2 .

$R^2 = 1$	$R^2 = 0$	$R^2 < 0$
Perfect prediction	Predicting the mean	Worse than predicting the mean

- Using pandas for data exploration.
- Clean the data by refilling the missing data with **SimpleImputer(strategy='mean')**. Removing the outliers with threshold **5000** which has the best result in R^2 .
- Feature scale** is critical to Lasso Regression. Larger scale features may dominate the penalty and distort features. Using **StandardScaler** for implementation.
- Model selection between **Lasso Regression** vs. **Ridge Regression**. Finalized with Lasso Regression due to L-1 penalty properties and zeroed out the weak coefficient features. This benefit also represented on the final R^2 score.
- Hyperparameter tuning with α -- **Strength Parameter**. Tested with different values for highest R^2 score, finalized with this Lasso model with using $\alpha = 5$. K-fold validation.

2. Step 1 — Exploratory Data Analysis

Dataset	Rows	Columns	Notes
Training Set	2,500	16	x0–x14 features + target column
Test Set	2,500	15	x0–x14 features only

Target Variable Statistics

Statistic	Value	Notes
Mean	-22.04	Negative shift center
Std	1,978.18	Extremely high variance
Min	-41,008.02	Severe negative outlier
Max	69,628.20	Severe positive outlier

Key observation: the target has extreme outliers. These extreme rows dominate the squared error and distort model training.

3. Step 2 — Data Cleaning

3a. Handling Missing Values (Imputation)

Both the training and test files contain missing values in every feature column.

Method: Using the average value of the column to fill the empty space in the table. `SimpleImputer(strategy='mean')`.

3b. Outlier Removal

Problem: Linear regression minimizes the sum of squared errors. Squaring amplifies extreme errors.

The extreme rows dominate the entire training signal. The solution is to remove them from training only.

```
OUTLIER_THRESHOLD = 5000
clean_mask = np.abs(y_train) <= OUTLIER_THRESHOLD

X_train_clean = X_train[clean_mask]
y_train_clean = y_train[clean_mask]
```

Threshold selection: 5 different thresholds were tested via cross-validation to find the optimal cutoff point.

Threshold	Rows Kept	Rows Removed	CV R ²
target > 2,000	2,444	56	0.0154
target > 3,000	2,461	39	0.0225
target > 4,000	2,469	31	0.0273
target > 5,000	2,476	24	0.0316 ← best
target > 7,000	2,485	15	0.0286
target > 10,000	2,490	10	0.0143

Result: 24 rows removed. 2,476 rows used for training. All 2,500 test rows are still predicted.

4. Step 3 — Feature Scaling for Lasso Regression

StandardScaler: transforms each feature to mean=0 and std=1.

Reason: Lasso penalizes coefficients based on their magnitude. The larger scale feature may dominate the penalty and distort feature selection.

- fit_transform on training data (learns the mean and std of each feature)
- transform only on test data (applies the same training statistics)

```
scaler = StandardScaler()
Xtr = scaler.fit_transform(X_train[tr_clean])
Xval = scaler.transform(X_train[val_idx])
```

5. Step 4 — Model Selection

5a. Ridge Regression vs. Lasso Regression

Ridge Regression and Lasso Regression models were tested using cross-validation with the same data cleaning pipeline (outlier threshold = 5,000):

Model	Best CV R ²	Notes
Ridge Regression	0.0310	Shrinks all coefficients, keeps all 15 features
Lasso Regression	0.0316	Shrinks weak features to zero -- auto feature selection

5b. Selected Lasso Regression

Lasso (Least Absolute Shrinkage and Selection Operator) uses an L1 penalty — it penalizes the sum of absolute coefficient values. This causes weak feature coefficients to shrink to exactly zero, effectively removing them from the model.

- Ridge (L2 penalty): shrinks all coefficients but never removes any feature
- Lasso (L1 penalty): sets weak coefficients to exactly zero; built-in feature selection

$$y = \beta_0 + \beta_1x_1 + \beta_2x_2 + \beta_3x_0 + \dots$$

On this dataset, many features have near-zero correlation with the target. Lasso automatically identified and removed 5 of them:

Feature	Coefficient	Status
x0	0.0000	Zeroed out
x1	0.0000	Zeroed out
x2	-6.2389	Kept
x3	-5.0738	Kept
x4	41.8812	Kept

x5	-19.2333	Kept
x6	5.4151	Kept
x7	0.0000	Zeroed out
x8	-11.0864	Kept
x9	90.9568	Kept — strongest signal
x10	0.4068	Kept
x11	0.0000	Zeroed out
x12	-0.5773	Kept
x13	0.0000	Zeroed out
x14	5.2090	Kept

Result: 10 of 15 features kept. 5 features (x0, x1, x7, x11, x13) zeroed out as irrelevant.

6. Step 5 — Hyperparameter Tuning (Alpha)

6a. Alpha (α)

Alpha is Lasso's regularization strength parameter -- a knob that controls how aggressively the model penalizes large coefficients.

$$Loss = Sum\ of\ Squared\ Errors + \alpha \sum_{j=1}^p |\beta_j|$$

Alpha	Effect	Risk
Too small	Fits training data closely	Overfitting
Just right	Learn real patterns	Best generalization
Too large	Model over-simplified	Underfitting

6b. Tuning Method — 5-Fold Cross-Validation

The training data is split into 5 equal folds. The model trains on 4 folds and validates on the remaining 1. This rotates 5 times so every row is validated exactly once. The alpha with the highest average R^2 is selected.

Alpha	CV R^2	Std Dev
0.1	0.0311	±0.0105
0.5	0.0311	±0.0106

1	0.0312	±0.0106
1.5	0.0313	±0.0107
2	0.0314	±0.0108
3	0.0315	±0.0109
5	0.0316 <- best	±0.0112
10	0.0304	±0.0111

```

kf = KFold(n_splits=5, shuffle=True, random_state=42)

best_alpha = None
best_cv_r2 = -np.inf

print("\nCross-validation R² - Lasso:")
for alpha in [0.1, 0.5, 1, 1.5, 2, 3, 5, 10]:
    r2s = []
    for tr_idx, val_idx in kf.split(X_train):
        tr_clean = tr_idx[np.abs(y_train[tr_idx]) <= OUTLIER_THRESHOLD]

        scaler = StandardScaler()
        Xtr = scaler.fit_transform(X_train[tr_clean])
        Xval = scaler.transform(X_train[val_idx])

        model = Lasso(alpha=alpha, max_iter=5000)
        model.fit(Xtr, y_train[tr_clean])
        r2s.append(r2_score(y_train[val_idx], model.predict(Xval)))

    mean_r2 = np.mean(r2s)
    std_r2 = np.std(r2s)
    print(f"  alpha={alpha:>5}  CV R² = {mean_r2:.4f}  (±{std_r2:.4f})")

    if mean_r2 > best_cv_r2:
        best_cv_r2 = mean_r2
        best_alpha = alpha

print(f"\nBest alpha: {best_alpha}  →  best CV R² = {best_cv_r2:.4f}")

```

Selected: $\alpha = 5$ with CV $R^2 = 0.0316$

7. Step 6 — Final Model & Predictions

- Final model trained on all 2,476 cleaned training rows using $\alpha = 5$
- StandardScaler fitted on clean training data, then applied to test data
- Predictions generated for all 2,500 test rows

8. Conclusion

Lasso Regression with data cleaning achieved a CV $R^2 = 0.0316$, making it the best single-model approach tested.

8a. Why R^2 is Low

The low R^2 reflects a genuinely difficult dataset:

- Target standard deviation $\approx 1,978$ (driven by extreme outliers up to $\pm 69,000$)
- Raw features have very weak linear correlations with the target (max 0.23)
- A linear model has fundamental limits on non-linear patterns